

Cloud Access Performance Testing

Overview

The objective of this report is for the RCVW development team to find the most robust C/C++ Open-Source message broker client tools while simultaneously gaining a perspective of the different protocols used to access the cloud.

Performance

The overall speed of delivery is considered to be the most important contributor to performance. Therefore, the tests tried to isolate individual steps in message processing so that each step could be measured more accurately. Thread synchronization was used to be able to 1) measure the true send-receive timing of each message, and 2) eliminate the use of wait “sleeps” that corrupt the understanding of the timings. Essentially, this keeps the machine busy queuing messages, instead of idling when waiting for each message to complete a round trip.

The round-trip timing will have at least 2 factors over which there is no direct control but which can be isolated empirically. These are 1) network speed, and 2) processing time at the endpoint between receive and send back.

To remove network speed as a factor, variable-sized messages are sent. Assuming that time on the wire would be somewhat linear with the data size, one can then estimate the amount of time the endpoint is using to process the message. This is achieved by subtracting out the “probable” communication time.

2 different protocols talking to 3 different endpoints were evaluated:

1. AMQP – Advanced Message Queuing Protocol
 - a. Azure Service Bus
 - b. Azure Event Hub
2. Kafka¹ (Apache Software Foundation, 2024)
 - a. Azure Event Hub configured with Kafka capabilities

Timing results

Figure 1 shows our measured round trip travel times for payload sizes from 1 to 20MB. From this, take the difference in seconds between a payload of 10 and 20 MB divided by 10 to calculate transmission time for 1 MB. The time it takes for the endpoint (server) to actually deal with the message must be the total time between send and receive *minus* the predicted travel time, which is (message size * transmission time per MB). Subtracting the test (payload size times this number) from the average round trip time (for that test) reveals the actual time the endpoint handled our message.

¹ Kafka is not a protocol per-se, but a “platform”

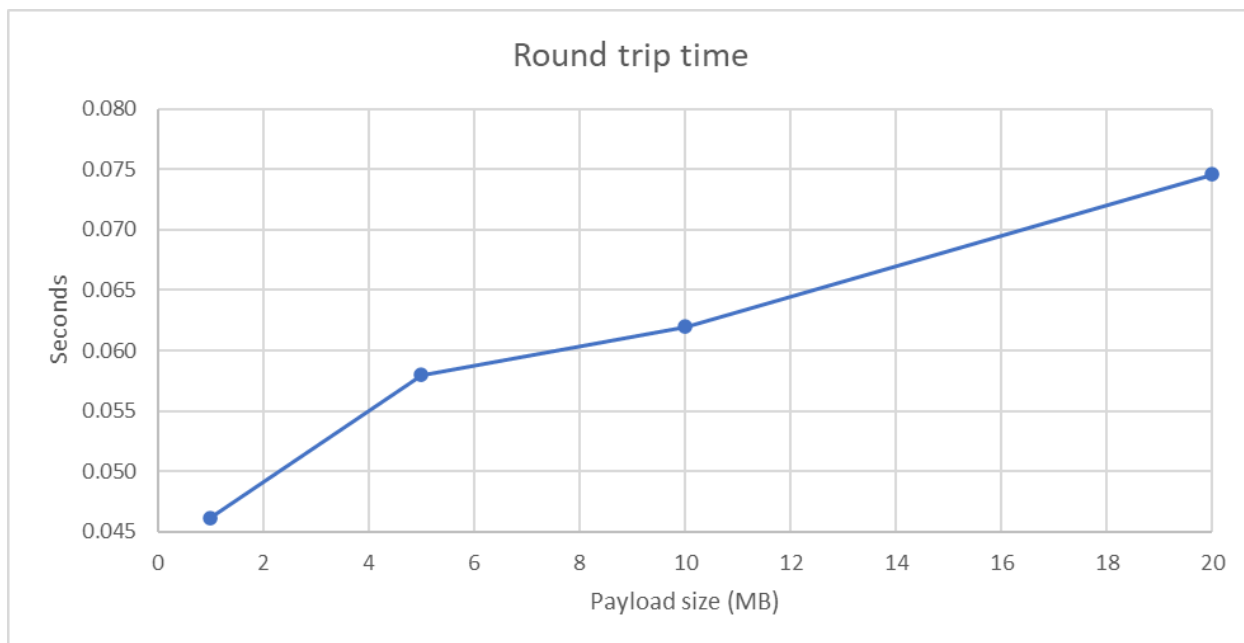


Figure 1 Round Trip second vs. Payload size

The calculation shows that processing messages using the AMQP protocol against the Azure Service bus endpoint happens at the rate of 0.05 seconds. For completeness, it was concluded that the AMQP protocol against the Azure *Event Hub* endpoint was close enough to determine there is no significant difference.

However, the test of Kafka against the Azure Event hub resulted in a factor of ~3 round trips around 0.120 seconds compared to those of 0.05 seconds.

Endpoint performance summary

Investigating Client APIs

Different client APIs or SDKs were studied to learn which might be best for current and future V2X-Hub prototype development.

For the AMQP protocol, 3 different API²s were investigated:

1. RabbitMQ, formerly called rabbitmq-c (Antonuk, 2024)
2. AMQP-CPP, a C++ RabbitMQ client (CopernicaMarketingSoftware, 2024)
3. Qpid-Proton and Apache project (Apache Software Foundation/Qpid Proton, 2024)

Rabbitmq-c

Upon connecting to an Azure Event Hub, the API misinterpreted the return string from the endpoint and threw an exception. Attempts were made to correct the functionality in order to get further into the connection process, however, a successful connection was not achieved.

² Keeping in mind that our client is C/C++ which limits the choices.

Additionally, this API is specifically targeted for the RabbitMQ message broker. Even though that broker is compatible with AMQP – it is not clear whether there is any difference between it and the Azure Event Hub/Service bus.

AMQP-CPP

The documentation for this API is better than most and is network agnostic in the sense that it does not actually implement I/O. It has abstract classes that require extension to create the socket or pipe connections to the endpoint. This opens the possibility of errors in a fairly complex custom component that would need to be built and maintained.

Qpid-Proton

Qpid-Proton's architecture is based on C++ and is implemented by extending virtual classes only for the functionality required, reducing the risks of errors. It provides meaningful diagnostics when there is an issue, such as invalid Azure SASL connection settings. The API is executed by means of message brokers and traditionally, clients use Containers for the connection aspects between themselves. The general object model of Qpid-Proton follows this nature more logically. All of this suggests a development pattern that can make for a very robust, and flexible, implementation. Specifically, its container object is the primary executor of all things from the client application that face the endpoint (message broker). Generally, one would create the container, and set it to "run" in its own thread. From there most, probably all, of the logic in the rest of the system is isolated from that container in its responsibilities (being pure message processing with the broker – *without* regard to application specific knowledge of what's in those messages). This works because the system has a thread-safe "work queue" which properly "injects" tasks for the attention of this container run-loop. That *could* essentially be the sole point of entry to messaging processing for the rest of the system – while the implementation around Qpid-Proton *could* be completely unaware of the logic feeding tasks into its work queue.

Conclusion

Pointing out that network time on the wire would be the same no matter what endpoint is used, the 3 times better performance of Azure Service Bus, or Event Hub is optimal. Additionally – the Event Hub and Service bus endpoints on Azure are roughly equivalent in terms of performance, which means that RCVW is able to use either one based on the type of messaging to perform (queue, topic, exchange) and need not mix performance concerns into choosing the approach.

Regarding the Client API's research, Qpid-Proton proved to be the best choice for the project needs. Implementing the cloud communications making use of the advantages that Qpid-Proton provides has significant benefits:

- The code communicating with the endpoint is completely independent of the client it is serving. In other words, it needs to know nothing about the logic in any of many systems it can serve, even simultaneously.
- Many systems never need "logic" within them that deals with access to the endpoints. That capability is handed completely over to a black box with simple interfaces. When capabilities improve and/or get fixed, there is no impact on the client(s).

- The communication to the endpoints can be offloaded to different processes, even different computers very simply as the needs arise.
- The type of environment and/or programming patterns fostered with distributed systems like this engender extremely robust, flexible, and efficient systems with the solid isolation of logic and capabilities from each other using well defined outbound and event interfaces.

References

1. Apache Software Foundation (2024). Introduction. Everything you need to know about Kafka in 10 minutes. <https://kafka.apache.org/intro>
2. Alan Antonuk, rabbitmq-c (2024). <https://github.com/alanxz/rabbitmq-c>
3. CopernicaMarketingSoftware, AMQP-CPP (2024). <https://github.com/CopernicaMarketingSoftware/AMQP-CPP>
4. Apache Software Foundation/Qpid Proton (2024). Qpid Proton. <https://qpid.apache.org/proton/>
5. Confluent Inc. (2024). <https://github.com/confluentinc/librdkafka>